Manual técnico — Finance AI

Nov 11, 2026 · @Michael

Introducción y alcance

Finance AI es una aplicación de finanzas personales para registrar gastos e ingresos, administrar cuentas y presupuestos, y capturar movimientos escribiendo o dictando lo que ocurrió. Este manual describe cómo está construida, cómo instalarla y cómo verificarla, en su versión en línea: el cliente necesita conexión con la API para operar.

El sistema se reparte en tres repositorios:

- **`finance-ai-backend`**: API FastAPI, modelos, migraciones de PostgreSQL, semillas, pruebas y el contrato `openapi.json`.
- **`finance-ai-frontend`**: cliente Flutter compartido para Android y Windows.
- **`finance-ai`**: documentación del proyecto y pruebas de integración entre cliente, API y base de datos.

La estructura de la base de datos se detalla en el documento *Diccionario de datos*.

Requisitos

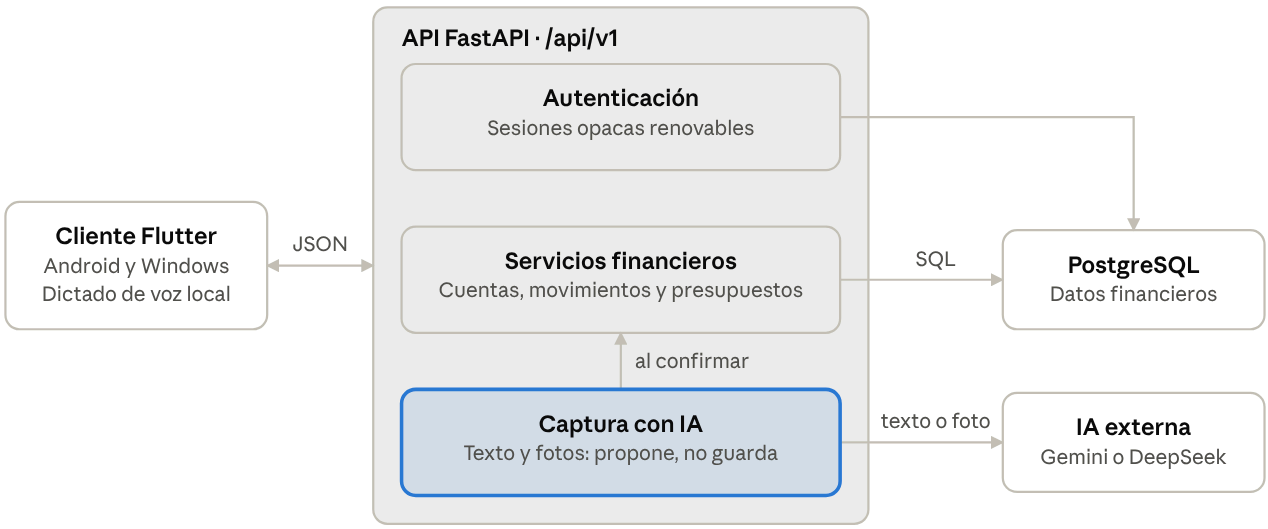
Para ejecutar el sistema completo en un equipo de desarrollo se necesita lo siguiente.

Componente	Requisito	Para qué se usa
Backend	Python 3.14 y el gestor `uv`	Instalar dependencias y ejecutar la API
Contenedores	Docker Desktop con Docker Compose	Levantar la API y PostgreSQL de forma aislada
Base de datos	PostgreSQL 18.6 (imagen `18.6-bookworm` de Compose)	Almacenar los datos financieros
Cliente	Flutter 3.47.2 con Dart 3.13.2	Compilar la app para Android y Windows

Componente	Requisito	Para qué se usa
Plataformas del cliente	Android (el dictado sin conexión exige Android 7 o superior) y Windows x64	Ejecutar la app
Proveedor de IA (opcional)	Clave de API de Gemini o de DeepSeek	Captura de gastos por texto, voz o foto; sin clave se usan los formularios manuales
Red	Acceso del cliente a la API	Registro, inicio de sesión y todas las operaciones financieras

Arquitectura

La captura con IA solo propone; el dato se guarda al confirmar



arquitectura · cliente, API, base de datos e IA

El cliente Flutter habla solo con la API. La API guarda en PostgreSQL y, cuando el usuario captura un movimiento por texto (escrito o dictado) o por foto, consulta a un proveedor de IA externo. La IA devuelve una propuesta; solo la confirmación del usuario crea el registro financiero.

Tecnologías

Cada capa usa una sola tecnología principal; las versiones son las declaradas en el código del proyecto.

Capa	Tecnología	Función
Interfaz	Flutter (Dart)	Un solo código de interfaz, estado y validación para Android y Windows
API	Python con FastAPI (0.140.x)	Servicio modular con rutas bajo `/api/v1` y contrato OpenAPI
Persistencia	SQLAlchemy 2.0 (2.0.45 o superior) y Alembic	Modelos, repositorios, unidades de trabajo y migraciones
Base de datos	PostgreSQL 18.6	Almacenamiento de usuarios, cuentas, transacciones, comprobantes y presupuestos
Inteligencia artificial	Gemini (por defecto) o DeepSeek, con respaldo entre ambos	Convertir texto o fotos de comprobantes en propuestas que el usuario revisa
Dictado por voz	Reconocimiento de voz local en el dispositivo	Pasar la voz a texto antes de enviarla al flujo de captura; el audio no sale del dispositivo
Autenticación	Credenciales opacas renovables, guardadas como hash en la base de datos	Sesiones de acceso de 15 minutos con renovación rotativa
Almacenamiento seguro	`flutter_secure_storage`	Guardar en el cliente la credencial de renovación
Contenedores	Docker Compose	Ejecutar localmente la API y PostgreSQL en una red privada
Calidad	pytest, ruff y mypy	Pruebas, estilo y verificación de tipos del backend

Instalación y configuración

Primero se levanta el backend con su base de datos y después se compila el cliente apuntando a la API.

Backend

Desde la carpeta `finance-ai-backend`, instalar dependencias y levantar la API junto con PostgreSQL en una red privada de Docker:

```
uv sync --all-groups --frozen
docker compose up --build -d
```

Con la base de datos disponible, aplicar las migraciones, cargar los catálogos iniciales (monedas, roles, métodos de pago y fuentes de ingreso) y, si se ejecuta sin Docker, iniciar la API:

```
uv run alembic upgrade head
uv run python -m finance_ai.db.seeds
uv run uvicorn finance_ai.main:app --reload
```

La captura con IA se configura con variables de entorno que permanecen en el servidor y nunca llegan al cliente:

Variable	Uso
`FINANCE_AI_LLM_PROVIDER`	Proveedor principal: `gemini` o `deepseek`
`FINANCE_AI_GEMINI_API_KEY`	Clave de Gemini
`FINANCE_AI_DEEPSEEK_API_KEY`	Clave de DeepSeek; si ambas claves existen, el otro proveedor actúa como respaldo

El archivo `.env` real no se sube al repositorio. El estado de la API se consulta en `/health/ready`.

Cliente Flutter

Desde la carpeta `finance-ai-frontend`, con Flutter 3.47.2, los scripts del proyecto verifican, compilan y prueban la app:

```
.\tool\verify_shared.ps1
.\tool\build_android.ps1
.\tool\test_windows_and_restore.ps1
```

Los resultados de depuración quedan en `build/app/outputs/flutter-apk/app-debug.apk` (Android) y `build/windows/x64/runner/Debug/finance_ai.exe` (Windows).

La dirección de la API se define al compilar o ejecutar con `--dart-define=FINANCE_AI_API_BASE_URL=<dirección>`. Si no se indica, el emulador de Android usa `http://10.0.2.2:8000` y Windows usa `http://127.0.0.1:8000`. En un teléfono físico hay que indicar la dirección del equipo en la red, o reenviar el puerto por USB con `adb reverse tcp:8000 tcp:8000`.

API

La API tiene trece grupos de rutas. Todas cuelgan de `/api/v1`, salvo las de salud. Excepto el registro, el inicio de sesión y la renovación, cada ruta exige una credencial de acceso y solo devuelve datos del usuario autenticado. El contrato completo está en el archivo `openapi.json` del backend.

Grupo	Ruta base	Operaciones
Salud	`/health/live`, `/health/ready`	Verificar que el servicio vive y que está listo
Autenticación	`/auth`	Registrar, iniciar sesión, renovar, consultar el usuario actual y cerrar sesión
Cuentas	`/accounts`	Crear, listar, consultar, editar, activar o desactivar (`/{id}/status`) y eliminar cuentas sin uso
Categorías	`/categories`	Crear, listar, ordenar (`PUT /order`), consultar, editar y eliminar
Fuentes de ingreso	`/income-sources`	Listar, crear, editar y eliminar

Grupo	Ruta base	Operaciones
Gastos	`/expenses`	Crear, listar, consultar, corregir y eliminar gastos
Ingresos	`/incomes`	Crear, listar, consultar, reemplazar y eliminar
Transferencias	`/transfers`	Crear una transferencia entre cuentas propias
Presupuestos	`/budgets`	Crear, listar, consultar, editar y eliminar, con el avance calculado al consultar
Historial	`/history`	Listar gastos, ingresos y transferencias con búsqueda, tipo, categoría, cuenta, periodo y paginación
Panel	`/dashboard/summary`	Resumen para la pantalla principal
Captura por texto	`/capture/text/proposals`	Proponer, consultar, descartar y confirmar un movimiento descrito en lenguaje natural
Comprobantes	`/receipts/proposal`, `/receipts/scan`	Proponer a partir de una foto, consultar, reintentar, descartar y confirmar; escaneo del comprobante

Seguridad y privacidad

El diseño parte de que ningún dato financiero se guarda sin que el usuario lo confirme y de que cada usuario solo ve lo suyo.

- **Sesiones:** la credencial de acceso dura 15 minutos y vive solo en la memoria de la app. La credencial de renovación rota en cada uso, vence a los 30 días sin actividad o a los 90 días desde el inicio, y se guarda en el cliente con `flutter_secure_storage`. Cada usuario puede tener hasta diez sesiones activas.
- **Credenciales en la base de datos:** solo se guardan hashes de los tokens, nunca su valor, y las contraseñas se almacenan como hash.

- **Aislamiento por usuario:** la propiedad de cada dato se deduce de la sesión. Un identificador inexistente y uno de otro usuario producen la misma respuesta, de modo que no se puede averiguar qué existe.
- **Captura con IA:** el modelo solo sugiere. Las propuestas son borradores privados que vencen a las 24 horas, y solo la confirmación explícita crea el gasto, ingreso o transferencia. Repetir la confirmación devuelve el primer resultado y no duplica el registro. Los mensajes de texto se limitan por usuario: una ráfaga de 6 y luego 20 por minuto.
- **Datos enviados a la IA:** las claves de los proveedores permanecen en el servidor. El audio no sale del dispositivo, y las fotos de comprobantes se procesan en memoria sin guardarse como archivo en el servidor.
- **Errores:** cada fallo público trae un código estable, un mensaje seguro y un identificador de correlación. Ni los errores ni los registros incluyen contraseñas, correos, credenciales, cuerpos de solicitudes ni SQL. En la app, el usuario puede copiar el informe del error para reportarlo.
- **Red de la base de datos:** PostgreSQL funciona en una red privada de Docker Compose.

Pruebas y calidad

El proyecto exige al menos 80 % de cobertura de líneas en backend y cliente. Estos son los comandos de verificación.

Ámbito	Comando	Qué comprueba
Backend	`uv run pytest --cov=finance_ai --cov-report=term-missing`	Pruebas unitarias y cobertura
Backend	`uv run ruff check .` y `uv run ruff format --check .`	Estilo y formato
Backend	`uv run mypy src`	Tipos
Backend	`docker compose --profile test run --rm test pytest -m integration`	Pruebas de integración con PostgreSQL real
Backend	`docker compose exec -T api alembic check`	Que los modelos y las migraciones coincidan

Ámbito	Comando	Qué comprueba
Cliente	`.\tool\verify_shared.ps1`	Verificación del código Dart compartido
Cliente	`.\tool\build_android.ps1`	Compilación de Android
Cliente	`.\tool\test_windows_and_restore.ps1`	Pruebas nativas de Windows, restaurando después la compilación normal
Sistema completo	`docker compose --profile integration run --rm flutter-integration` (desde `finance-ai`)	Registro, inicio de sesión, restauración de sesión y cierre de sesión contra una API y una base de datos de prueba aisladas

Las pruebas de integración del backend bajan la base de datos a su estado inicial, la suben a la última migración y cargan los catálogos. Se niegan a correr sobre cualquier base cuyo nombre no termine en `_test`, para no tocar datos reales.

Despliegue y limitaciones

En desarrollo, Docker Compose levanta la API y PostgreSQL en una red privada. El repositorio del backend incluye además un flujo de despliegue a Google Cloud Run (`.github/workflows/deploy.yml`); su existencia no indica por sí sola que haya un despliegue activo.

Limitaciones conocidas de esta versión:

- **Conexión obligatoria:** el cliente necesita llegar a la API para registrarse, iniciar sesión y operar. No hay almacenamiento local de datos financieros ni cola de sincronización.
- **Plataformas:** el cliente está disponible para Android y Windows.
- **Dependencia de un proveedor de IA:** si ningún proveedor responde, la captura por texto devuelve `503 text_capture_unavailable` y el usuario sigue con los formularios manuales.
- **Imágenes de comprobantes:** el servidor no las guarda como archivo; la app conserva localmente, cifrada, la foto del comprobante confirmado.
- **Dictado sin conexión en Android:** requiere descargar una sola vez el modelo del idioma, de unos 40 MB.

- **Asistente conversacional:** consultas en lenguaje natural sobre los datos (chat con IA) quedaron como mejora opcional futura y no están implementadas.